Reporting des résultats du PoC MedHead

1. Objet du document

Ce document présente les résultats obtenus lors de la réalisation et de la validation du Proof of Concept MedHead. Il synthétise les fonctionnalités démontrées, les résultats des tests automatisés et des tests de performance, les limites observées ainsi que les enseignements à retenir avant une éventuelle industrialisation de la solution.

Le reporting s'appuie sur les exigences, hypothèses et principes architecturaux définis dans les documents existants du repository architectural.

Repository du code du PoC : https://github.com/naegel23/P11-MedHead-PoC

2. Résultats fonctionnels

Le PoC permet de rechercher des établissements hospitaliers à partir de la localisation d'une urgence, d'une spécialité médicale et d'un rayon de recherche.

Le backend, modernisé sous Java 17 et Spring Boot 3.5.16, expose notamment les fonctionnalités suivantes :

- consultation des établissements hospitaliers ;
- filtrage par spécialité ;
- filtrage selon la disponibilité des lits ;
- filtrage géographique dans un rayon défini ;
- recommandation d'établissements à partir du calcul de distance et de temps de trajet ;
- exposition des fonctionnalités au travers d'une API REST documentée avec OpenAPI 3 via springdoc.

La spécification OpenAPI est disponible sur **/v3/api-docs** et l'interface Swagger UI sur **/swagger-ui/index.html**.

Une interface Vue.js a également été intégrée au PoC afin de permettre la saisie des paramètres de recherche et l'affichage des établissements retournés par l'API.

3. Résultats des tests et de l'intégration continue

Les différentes campagnes de validation ont confirmé le bon fonctionnement du PoC sur les périmètres testés.

Backend

Les tests Maven ont été exécutés avec succès, notamment les tests unitaires et les tests d'intégration associés au backend Spring Boot. La commande **mvn clean verify -Pfailsafe** termine sans erreur.

Frontend

Le frontend Vue.js dispose de tests automatisés exécutés avec Vitest et Vue Test Utils. Les tests vérifient notamment l'affichage du formulaire et le comportement de la recherche avec une réponse simulée du backend.

Intégration continue

Le workflow GitHub Actions MedHead CI est exécuté lors des push et pull requests vers la branche **main**. Il vérifie le backend et le frontend, exécute les tests, construit les artefacts et publie les rapports utiles au contrôle du PoC.

Le workflow a été aligné avec la stack modernisée : Java 17, actions GitHub maintenues et runner Ubuntu 24.04. Les exécutions réalisées après les principales évolutions du projet ont terminé avec succès.

Conteneurisation

L'image Docker du backend, basée sur un runtime Java 17, a été construite et exécutée avec succès. Une requête HTTP effectuée vers l'API exécutée dans le conteneur a retourné une réponse HTTP 200, confirmant que le service peut être lancé de manière conteneurisée.

4. Résultats des tests de performance

Les tests de charge ont été réalisés avec Apache JMeter afin d'évaluer le comportement du backend sous sollicitation.

Un scénario de référence a été exécuté sur l'endpoint de recherche des hôpitaux disponibles par spécialité et rayon, sans inclure la dépendance de routage externe. Le scénario est configuré pour viser 800 requêtes par seconde.

Résultat du test de référence

- Requêtes exécutées : 100 000
- Utilisateurs virtuels : 100
- Débit effectivement mesuré : 998.93 requêtes par seconde
- Taux d'erreur : 0 %
- Temps de réponse moyen : 3.98 ms
- Médiane : 2 ms
- P90 : 5 ms
- P95 : 15.95 ms
- P99 : 50 ms
- Temps maximum observé : 137 ms

Dans les conditions de ce run, le débit de référence de 800 requêtes par seconde est dépassé sans erreur et l'ensemble des indicateurs de latence observés reste sous le seuil de 200 ms, y compris le maximum mesuré.

Le PoC valide donc le critère de performance sur cet endpoint dans les conditions de test locales. Ces résultats ne constituent toutefois pas à eux seuls une garantie de performance en environnement de production, qui devra être validée sur une infrastructure représentative.

5. Limites et écarts observés

Dépendance au service de routage

Le scénario de bout en bout **/nearest/...** dépend du service externe OpenRouteService pour le calcul des itinéraires. Lors des essais sous charge, cette dépendance a retourné des réponses HTTP 429 Too Many Requests. Dans ce cas, le backend peut retourner une réponse 408 Request Timeout.

Cette dépendance constitue donc un risque pour la disponibilité et la performance du service cible. Une industrialisation devra prévoir un quota adapté, une stratégie de retry avec backoff, éventuellement du cache et un service de routage disposant d'un niveau de service compatible avec les besoins de MedHead.

Sécurité

Le PoC ne manipule pas de données personnelles de patients. Les mécanismes complets de sécurité de production, notamment authentification, autorisation et HTTPS, ne sont pas implémentés dans le périmètre actuel et devront être ajoutés avant une mise en production.

Architecture cible

Le PoC valide un bloc fonctionnel de recommandation hospitalière. Il ne constitue pas à lui seul l'architecture microservices complète définie dans l'architecture cible MedHead. La publication d'événements et l'intégration à un éventuel bus de messages appartiennent aux étapes ultérieures de l'industrialisation.

KPI métier

Les objectifs métier relatifs au taux d'orientation des urgences vers un établissement compétent et à la réduction du délai moyen de prise en charge nécessitent des données et des processus opérationnels réels. Le PoC démontre la faisabilité technique de la recherche et de la recommandation, mais ne permet pas à lui seul de conclure à l'atteinte de ces KPI en production.

6. Recommandations avant industrialisation

Avant une mise en production, plusieurs actions sont recommandées :

- renforcer la gestion de la dépendance au service de routage avec retry, backoff, cache et supervision des quotas ;
- mettre en place les mécanismes de sécurité nécessaires, notamment authentification, autorisation, gestion des secrets et HTTPS ;
- poursuivre les tests de performance sur une infrastructure représentative de la production afin de confirmer les résultats observés localement ;
- continuer à surveiller les percentiles P95 et P99 lors des futures campagnes de charge et en exploitation ;
- intégrer le service dans l'architecture cible MedHead, notamment avec les composants d'infrastructure et les mécanismes d'échange prévus par la feuille de route ;
- mettre en place l'observabilité nécessaire en production : métriques, logs centralisés, alertes et suivi des erreurs ;
- conserver la chaîne CI et l'automatisation des tests comme base du processus de livraison continue.

7. Conclusion

Le PoC MedHead démontre la faisabilité technique du service de recommandation hospitalière sur le périmètre étudié. Les fonctionnalités principales sont opérationnelles, les tests automatisés et la chaîne CI sont fonctionnels, le backend est conteneurisable et le scénario de performance de référence dépasse le niveau de 800 requêtes par seconde sans erreur avec des latences observées inférieures à 200 ms sur le run de référence.

La modernisation vers Java 17, Spring Boot 3 et OpenAPI 3 permet en outre d'aligner le PoC sur une stack plus actuelle tout en conservant son comportement fonctionnel validé.

Les essais mettent également en évidence plusieurs sujets à traiter avant une mise en production, en particulier la dépendance au service de routage externe, la sécurité, l'observabilité et l'intégration dans l'architecture cible.

Le PoC fournit ainsi des éléments favorables à la poursuite du projet, sous réserve de traiter ces points lors des prochaines étapes d'industrialisation.